

LAPORAN UTS PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

Disusun untuk Memenuhi Tugas Mata Kuliah Pemrograman Berorientasi Objek

Dosen Pengampu:

Mutaqin Akbar, S.Kom., M.T., MCE., MCF.



Disusun oleh :

Nama : DIMAS NURDIANSYAH
NIM : 251110058

UNIVERSITAS MERCU BUANA YOGYAKARTA
FAKULTAS TEKNOLOGI INFORMASI
PROGRAM STUDI INFORMATIKA
2025/2026

DAFTAR ISI

DAFTAR ISI	ii
DAFTAR GAMBAR	iii
A. Tugas	1
B. Jawaban.....	2
1. Rancangan Program	2
2. Source Code	3
2.1 Class Mesin.java	3
2.2 Class SepedaMotor.java	5
2.3 Class Mobil.java.....	7
2.4 Class Utama.java	8
3. Output.....	11
4. Kesimpulan.....	14

DAFTAR GAMBAR

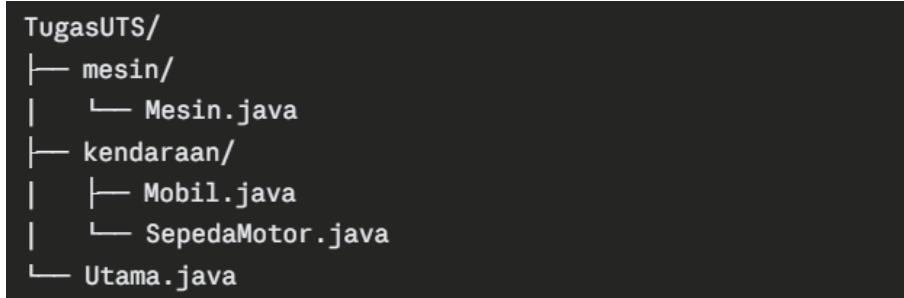
Gambar 2. 1 Struktur Program.....	2
Gambar 2.1. 1 Source Code Class Mesin.....	3
Gambar 2.2. 1 Source Code Class Sepeda Motor.....	5
Gambar 2.3. 1 Source Code Class Mobil.....	7
Gambar 2.4. 1 Source Code Class Utama.....	8
Gambar 3. 1 Output Program.....	11

A. Tugas

1. Buatlah sebuah program JAVA dengan rincian sebagai berikut:
 - a. Buatlah kelas Mesin dengan atribut jumlah silinder, kapasitas silinder dan langkah (silakan pilih hak akses yang dapat diakses oleh kelas lain pada soal ini).
 - b. Buatlah kelas SepedaMotor dengan:
 - Atribut bahan bakar
 - Atribut mesin (mesin merupakan objek dari kelas Mesin)
 - Tambahkan 1 buah atribut selain yang disebut diatas
 - Semua atribut memiliki hak akses private
 - Buatlah method getter dan setter untuk setiap atribut diatas!
 - c. Buatlah kelas Mobil dengan :
 - Atribut bahan bakar
 - Atribut mesin (mesin merupakan objek dari kelas Mesin)
 - Tambahkan 1 buah atribut selain yang disebut diatas
 - Semua atribut memiliki hak akses private
 - Buatlah method getter dan setter untuk setiap atribut diatas!
 - d. Masukkan kelas Mesin pada sebuah package bernama mesin!
 - e. Masukkan kelas SepedaMotor dan Mobil pada sebuah package bernama kendaraan!
 - f. Buatlah kelas Utama diluar package kendaraan dan mesin, kemudian buatlah objek kelas SepedaMotor dan Mobil masing-masing satu dan cetaklah atribut dari masing-masing objek!
2. Jelaskan semua source code yang sudah saudara buat!

B. Jawaban

1. Rancangan Program



Gambar 2.1 Struktur Program

Program yang saya buat ini terdiri dari empat kelas yang dibagi ke dalam dua package dan satu default package. Kelas pertama adalah Mesin yang berada di dalam package mesin. Kelas ini memiliki tiga atribut yaitu jumlahSilinder bertipe int untuk menyimpan jumlah silinder mesin, kapasitasSilinder bertipe double untuk menyimpan kapasitas silinder dalam satuan CC, dan langkah bertipe String untuk menyimpan jenis langkah mesin seperti 2-Tak atau 4-Tak. Kelas Mesin dilengkapi dengan sebuah constructor serta getter dan setter untuk setiap atributnya.

Kelas kedua dan ketiga berada di dalam package kendaraan yaitu kelas SepedaMotor dan kelas Mobil. Kelas SepedaMotor memiliki atribut bahanBakar bertipe String untuk menyimpan jenis bahan bakar, mesin bertipe objek Mesin sebagai spesifikasi mesin sepeda motor, dan satu atribut tambahan yaitu merk bertipe String untuk menyimpan nama merk kendaraan. Kelas Mobil memiliki atribut bahanBakar bertipe String, mesin bertipe objek Mesin, dan satu atribut tambahan yaitu jumlahPintu bertipe int untuk menyimpan jumlah pintu mobil. Semua atribut pada kedua kelas tersebut bersifat private dan masing-masing dilengkapi dengan getter dan setter sebagai penerapan konsep enkapsulasi.

Kelas keempat adalah Utama yang berada di luar kedua package sebagai entry point atau titik masuk program. Kelas ini bertugas membuat objek SepedaMotor dan Mobil, mengisi nilai atributnya menggunakan setter, serta mencetak seluruh data dari masing-masing objek ke layar menggunakan System.out.println.

2. Source Code

2.1 Class Mesin.java

```
mesin >  Mesin.java
1  package mesin;
2
3  public class Mesin {
4
5      private int    jumlahSilinder;
6      private double kapasitasSilinder;
7      private String langkah;
8
9      public Mesin(int jumlahSilinder, double kapasitasSilinder, String langkah) {
10         this.jumlahSilinder    = jumlahSilinder;
11         this.kapasitasSilinder = kapasitasSilinder;
12         this.langkah           = langkah;
13     }
14
15     public int getJumlahSilinder() {
16         return jumlahSilinder;
17     }
18     public void setJumlahSilinder(int jumlahSilinder) {
19         this.jumlahSilinder = jumlahSilinder;
20     }
21
22     public double getKapasitasSilinder() {
23         return kapasitasSilinder;
24     }
25     public void setKapasitasSilinder(double kapasitasSilinder) {
26         this.kapasitasSilinder = kapasitasSilinder;
27     }
28
29     public String getLangkah() {
30         return langkah;
31     }
32     public void setLangkah(String langkah) {
33         this.langkah = langkah;
34     }
35 }
```

Gambar 2.1. 1 Source Code Class Mesin

Penjelasan Kode Program :

Class Mesin adalah kelas yang merepresentasikan spesifikasi mesin kendaraan. Kelas ini berdiri sendiri dan tidak mewarisi kelas lain, namun digunakan sebagai tipe atribut (objek) di dalam kelas SepedaMotor dan Mobil.

Penjelasan baris per baris:

- `package mesin;` → Baris ini mendeklarasikan bahwa file Mesin.java termasuk ke dalam package bernama mesin. Package berfungsi seperti folder pengelompokan kelas agar terorganisir dan tidak bentrok dengan kelas lain yang memiliki nama sama di package berbeda.
- `private int jumlahSilinder;` → Atribut ini menyimpan jumlah silinder pada mesin kendaraan. Tipe data int (integer) digunakan karena jumlah silinder selalu berupa bilangan bulat, misalnya 1, 2, atau 4. Access modifier private berarti atribut ini

hanya bisa diakses dari dalam class Mesin saja, tidak bisa diakses langsung dari luar kelas sebagai penerapan konsep Enkapsulasi.

- `private double kapasitasSilinder;` → Atribut ini menyimpan kapasitas silinder dalam satuan CC (Cubic Centimeter). Tipe data double digunakan karena kapasitas silinder bisa berupa bilangan desimal, misalnya 150.0 CC atau 1500.0 CC. Sama seperti sebelumnya, private menjaga agar data tidak bisa diakses sembarangan dari luar kelas.
- `private String langkah;` → Atribut ini menyimpan jenis langkah mesin, misalnya "2-Tak" atau "4-Tak". Tipe data String digunakan karena nilainya berupa teks. Access modifier private diterapkan konsisten untuk semua atribut.
- `public Mesin (int jumlahSilinder, double kapasitasSilinder, String langkah)` → Ini adalah Constructor dari kelas Mesin. Constructor adalah method khusus yang dipanggil secara otomatis saat objek Mesin baru dibuat menggunakan keyword new. Constructor ini menerima tiga parameter sesuai atribut yang dimiliki kelas. Keyword `this.jumlahSilinder = jumlahSilinder` digunakan untuk membedakan antara atribut milik kelas (`this.jumlahSilinder`) dengan parameter yang masuk (`jumlahSilinder`), karena keduanya memiliki nama yang sama.

Getter dan Setter, Mengapa ditambahkan?

- Karena semua atribut bersifat `private`, maka tidak ada cara mengakses atau mengubah nilainya dari luar kelas secara langsung. Oleh karena itu ditambahkan getter dan setter sebagai pintu akses resmi:
- `public int getJumlahSilinder()` → method untuk membaca nilai `jumlahSilinder` dari luar kelas. Bertipe `int` sesuai tipe atributnya.
- `public void setJumlahSilinder(int jumlahSilinder)` → method untuk mengubah nilai `jumlahSilinder` dari luar kelas. Bertipe `void` karena tidak mengembalikan nilai, hanya menyimpan.
- Hal yang sama berlaku untuk `getKapasitasSilinder()` / `setKapasitasSilinder()` bertipe `double`, dan `getLangkah()` / `setLangkah()` bertipe `String`. Meskipun soal tidak secara eksplisit meminta getter/setter pada kelas Mesin, getter/setter tetap ditambahkan karena atribut dibuat `private`. Ini konsisten dengan prinsip Enkapsulasi dalam OOP data dilindungi dari akses langsung, dan hanya bisa dibaca atau diubah melalui method yang telah disediakan. Hal ini juga membuat kode lebih konsisten dengan kelas SepedaMotor dan Mobil yang juga menerapkan pola yang sama.

2.2 Class SepedaMotor.java

```
kendaraan > SepedaMotor.java
1  package kendaraan;
2
3  import mesin.Mesin;
4
5  public class SepedaMotor {
6
7      private String bahanBakar;
8      private Mesin mesin;
9      private String merk;
10
11     public String getBahanBakar() {
12         return bahanBakar;
13     }
14     public void setBahanBakar(String bahanBakar) {
15         this.bahanBakar = bahanBakar;
16     }
17
18     public Mesin getMesin() {
19         return mesin;
20     }
21     public void setMesin(Mesin mesin) {
22         this.mesin = mesin;
23     }
24
25     public String getMerk() {
26         return merk;
27     }
28     public void setMerk(String merk) {
29         this.merk = merk;
30     }
31 }
```

Gambar 2.2. 1 Source Code Class Sepeda Motor

Penjelasan Kode Program :

Class SepedaMotor merepresentasikan entitas sepeda motor yang memiliki bahan bakar, mesin, dan merk. Kelas ini berada di package kendaraan bersama kelas Mobil.

Penjelasan baris per baris:

- `package kendaraan;` → Mendeklarasikan bahwa file ini berada di dalam package kendaraan. Package ini mengelompokkan semua kelas yang berhubungan dengan jenis kendaraan.
- `import mesin.Mesin;` → Karena kelas SepedaMotor menggunakan objek dari kelas Mesin yang berada di package berbeda (mesin), maka harus di-import terlebih dahulu menggunakan keyword import. Tanpa baris ini, Java tidak akan mengenali tipe data Mesin.
- `private String bahanBakar;` → Atribut ini menyimpan jenis bahan bakar sepeda motor, misalnya "Pertalite" atau "Pertamax". Tipe data String digunakan karena nilainya berupa teks. private diterapkan agar data hanya bisa diakses melalui getter/setter.

- `private Mesin mesin;` → Ini adalah atribut yang sangat penting tipe datanya bukan tipe primitif, melainkan sebuah objek dari kelas Mesin. Inilah yang disebut konsep Object Composition (komposisi objek), yaitu sebuah kelas memiliki kelas lain sebagai atributnya. `private` diterapkan sehingga objek mesin hanya bisa diakses melalui getter/setter.
- `private String merk;` → Atribut tambahan yang dipilih sebagai satu atribut ekstra sesuai ketentuan soal. Menyimpan nama merk sepeda motor, misalnya "Yamaha" atau "Honda". Bertipe String karena berupa teks.
- `public String getBahanBakar()` → mengembalikan nilai bahanBakar bertipe String.
- `public void setBahanBakar(String bahanBakar)` → mengubah nilai bahanBakar, bertipe void.
- `public Mesin getMesin()` → mengembalikan objek mesin bertipe Mesin. Getter ini sangat penting karena melalui getter inilah kelas lain bisa mengakses atribut-atribut di dalam objek mesin.
- `public void setMesin(Mesin mesin)` → menerima objek Mesin dan menyimpannya ke atribut mesin.
- `public String getMerk()` → mengembalikan nilai merk bertipe String.
- `public void setMerk(String merk)` → mengubah nilai merk, bertipe void.

2.3 Class Mobil.java

```
kendaraan > Mobil.java
1  package kendaraan;
2
3  import mesin.Mesin;
4
5  public class Mobil {
6
7      private String bahanBakar;
8      private Mesin mesin;
9      private int jumlahPintu;
10
11     public String getBahanBakar() {
12         return bahanBakar;
13     }
14     public void setBahanBakar(String bahanBakar) {
15         this.bahanBakar = bahanBakar;
16     }
17
18     public Mesin getMesin() {
19         return mesin;
20     }
21     public void setMesin(Mesin mesin) {
22         this.mesin = mesin;
23     }
24
25     public int getJumlahPintu() {
26         return jumlahPintu;
27     }
28     public void setJumlahPintu(int jumlahPintu) {
29         this.jumlahPintu = jumlahPintu;
30     }
31 }
```

Gambar 2.3. 1 Source Code Class Mobil

Penjelasan Kode Program :

Class Mobil merepresentasikan entitas mobil yang memiliki bahan bakar, mesin, dan jumlah pintu. Strukturnya sangat mirip dengan SepedaMotor, namun atribut tambahannya berbeda.

Penjelasan baris per baris:

- `package kendaraan;` → Sama dengan SepedaMotor, file ini berada di package kendaraan sehingga keduanya dapat saling diakses tanpa import.
- `import mesin.Mesin;` → Sama seperti SepedaMotor, kelas Mobil juga menggunakan objek Mesin sebagai atribut, sehingga harus di-import dari package mesin.
- `private String bahanBakar;` → Atribut ini menyimpan jenis bahan bakar mobil, misalnya "Pertamax". Tipe data String digunakan karena berupa teks.
- `private Mesin mesin;` → Sama seperti di SepedaMotor, atribut ini bertipe objek Mesin yang merupakan penerapan Object Composition. Satu objek Mesin yang berbeda dibuat untuk mobil dengan spesifikasi yang berbeda pula.

- `private int jumlahPintu;` → Atribut tambahan yang dipilih untuk kelas Mobil. Menyimpan jumlah pintu pada mobil, misalnya 2 atau 4. Tipe data int dipilih karena jumlah pintu selalu berupa bilangan bulat.
- `public String getBahanBakar()` → mengembalikan nilai bahanBakar bertipe String.
- `public void setBahanBakar(String bahanBakar)` → mengubah nilai bahanBakar.
- `public Mesin getMesin()` → mengembalikan objek mesin bertipe Mesin.
- `public void setMesin(Mesin mesin)` → menyimpan objek Mesin ke atribut mesin.
- `public int getJumlahPintu()` → mengembalikan nilai jumlahPintu bertipe int.
- `public void setJumlahPintu(int jumlahPintu)` → mengubah nilai jumlahPintu, bertipe void.

2.4 Class Utama.java

```

1  import kendaraan.Mobil;
2  import kendaraan.SepedaMotor;
3  import mesin.Mesin;
4
5  public class Utama {
6
7      public static void main(String[] args) {
8
9          System.out.println("Nama      : Dimas Nurdiansyah");
10         System.out.println("NIM       : 251110058\n");
11         System.out.println("DEALER  JOGJA ISTIMEWA");
12         System.out.println("=".repeat(40));
13
14         Mesin mesinMotor = new Mesin(1, 150.0, "2-Tak");
15
16         SepedaMotor motor = new SepedaMotor();
17         motor.setBahanBakar("Pertalite");
18         motor.setMesin(mesinMotor);
19         motor.setMerk("Yamaha");
20
21         System.out.println("DATA SEPEDA MOTOR");
22         System.out.println("Merk          : " + motor.getMerk());
23         System.out.println("Bahan Bakar    : " + motor.getBahanBakar());
24         System.out.println("Spesifikasi Mesin :");
25         System.out.println("Jumlah Silinder : " + motor.getMesin().getJumlahSilinder());
26         System.out.println("Kapasitas Silinder : " + motor.getMesin().getKapasitasSilinder() + " CC");
27         System.out.println("Langkah        : " + motor.getMesin().getLangkah());
28
29         System.out.println();
30
31         Mesin mesinMobil = new Mesin(4, 1500.0, "4-Tak");
32
33         Mobil mobil = new Mobil();
34         mobil.setBahanBakar("Pertamax");
35         mobil.setMesin(mesinMobil);
36         mobil.setJumlahPintu(4);
37
38         System.out.println("DATA MOBIL");
39         System.out.println("Jumlah Pintu    : " + mobil.getJumlahPintu());
40         System.out.println("Bahan Bakar     : " + mobil.getBahanBakar());
41         System.out.println("Spesifikasi Mesin :");
42         System.out.println("Jumlah Silinder : " + mobil.getMesin().getJumlahSilinder());
43         System.out.println("Kapasitas Silinder : " + mobil.getMesin().getKapasitasSilinder() + " CC");
44         System.out.println("Langkah         : " + mobil.getMesin().getLangkah());
45     }
46 }

```

Gambar 2.4. 1 Source Code Class Utama

Penjelasan Kode Program :

Package: default (tidak memiliki package, berada di luar folder mesin dan kendaraan)
Class Utama adalah entry point atau titik masuk program. Di sinilah seluruh objek dibuat, dihubungkan, dan outputnya dicetak ke layar.

Penjelasan Kode Program:

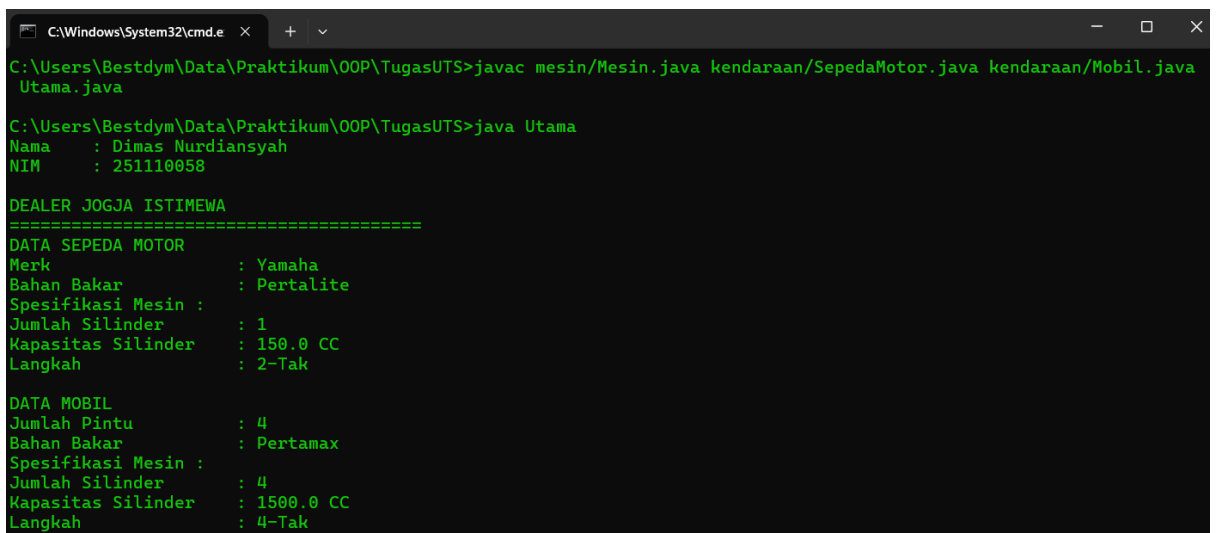
- `import kendaraan.Mobil; import kendaraan.SepedaMotor; import mesin.Mesin;` → Karena Utama.java berada di luar semua package, maka semua kelas yang digunakan harus di-import satu per satu. Tanpa ketiga baris import ini, Java tidak akan mengenali kelas Mobil, SepedaMotor, maupun Mesin.
- `public static void main(String[] args)` → Method utama yang dijalankan pertama kali oleh JVM (Java Virtual Machine) saat program dieksekusi. static berarti method ini bisa dijalankan tanpa membuat objek Utama terlebih dahulu. String[] args adalah parameter untuk menampung argumen dari command line, namun tidak digunakan pada program ini.
- `System.out.println("Nama : Dimas Nurdiansyah"); System.out.println("NIM : 251110058");` → Mencetak identitas diri saya ke layar. System.out.println adalah perintah bawaan Java untuk mencetak teks ke layar dan otomatis pindah baris setelah teks selesai dicetak.
- `System.out.println("DEALER JOGJA ISTIMEWA"); System.out.println("=".repeat(40));` → Mencetak nama dealer sebagai header program. "=".repeat(40) adalah method bawaan Java untuk mengulang karakter = sebanyak 40 kali sehingga menghasilkan garis pemisah yang rapi.
- `Mesin mesinMotor = new Mesin(1, 150.0, "2-Tak");` → Membuat objek Mesin khusus untuk sepeda motor menggunakan Constructor Mesin yang menerima tiga parameter: jumlahSilinder (1), kapasitasSilinder (150.0), dan langkah ("2-Tak"). Objek ini disimpan ke variabel mesinMotor bertipe Mesin.
- `SepedaMotor motor = new SepedaMotor();` → Membuat objek SepedaMotor tanpa parameter menggunakan default constructor yang otomatis dibuat Java karena tidak ada constructor eksplisit di kelas SepedaMotor. Objek disimpan ke variabel motor.
- `motor.setBahanBakar("Pertalite");` → Mengisi atribut bahanBakar pada objek motor dengan nilai "Pertalite" menggunakan setter. Nilai bertipe String sehingga ditulis dengan tanda kutip.
- `motor.setMesin(mesinMotor);` → Menyimpan objek mesinMotor yang sudah dibuat sebelumnya ke dalam atribut mesin pada objek motor menggunakan setter. Inilah yang menghubungkan objek Mesin dengan objek SepedaMotor.
- `motor.setMerk("Yamaha");` → Mengisi atribut tambahan merk pada objek motor dengan nilai "Yamaha". Atribut merk merupakan 1 atribut tambahan yang ditambahkan pada kelas SepedaMotor sesuai ketentuan soal, selain atribut bahanBakar dan mesin yang sudah diwajibkan.
- `System.out.println("DATA SEPEDA MOTOR");` → Mencetak teks header sebagai judul bagian data sepeda motor.

- `System.out.println("Merk : " + motor.getMerk());` → Mencetak merk sepeda motor. Tanda + digunakan untuk menggabungkan (concatenation) antara teks statis "Merk : " dengan nilai yang dikembalikan oleh `motor.getMerk()`. Method `getMerk()` dipanggil pada objek motor untuk mengambil nilai atribut merk yang bersifat private. Hasilnya tercetak Merk : Yamaha.
- `System.out.println("Bahan Bakar : " + motor.getBahanBakar());` → Mencetak jenis bahan bakar. `motor.getBahanBakar()` memanggil getter pada objek motor untuk mengambil nilai atribut bahanBakar. Hasilnya tercetak Bahan Bakar : Pertalite.
- `System.out.println("Spesifikasi Mesin :");` → Mencetak teks sub-header sebagai penanda bahwa baris-baris berikutnya adalah data spesifikasi mesin.
- `System.out.println("Jumlah Silinder : " + motor.getMesin().getJumlahSilinder());` → Baris ini menggunakan method chaining pemanggilan method secara berantai dalam satu baris. Pertama `motor.getMesin()` dipanggil untuk mendapatkan objek Mesin yang tersimpan di dalam objek motor. Kemudian dari objek Mesin tersebut langsung dipanggil `.getJumlahSilinder()` untuk mengambil nilai jumlah silinder. Cara ini bisa dilakukan karena atribut mesin di kelas `SepedaMotor` bersifat private namun memiliki getter, dan atribut `jumlahSilinder` di kelas `Mesin` juga bersifat private namun memiliki getter tersendiri. Hasilnya tercetak Jumlah Silinder : 1.
- `System.out.println("Kapasitas Silinder : " + motor.getMesin().getKapasitasSilinder() + " CC");` → Sama seperti sebelumnya menggunakan method chaining untuk mengambil kapasitas silinder. Tanda + " CC" di akhir menambahkan satuan setelah angka kapasitas. Hasilnya tercetak Kapasitas Silinder : 150.0 CC. Nilai tampil sebagai 150.0 karena tipe datanya double.
- `System.out.println("Langkah : " + motor.getMesin().getLangkah());` → Mengambil nilai atribut langkah dari objek Mesin milik motor melalui method chaining. Hasilnya tercetak Langkah : 2-Tak.
- `System.out.println();` → Mencetak baris kosong tanpa teks apapun, berfungsi sebagai jarak pemisah antara blok data sepeda motor dan blok data mobil agar output lebih rapi dan mudah dibaca.
- `Mesin mesinMobil = new Mesin(4, 1500.0, "4-Tak");` → Membuat objek Mesin baru yang berbeda khusus untuk mobil. Parameter yang diberikan adalah `jumlahSilinder` (4), `kapasitasSilinder` (1500.0), dan `langkah` ("4-Tak"). Objek ini terpisah dari `mesinMotor` karena spesifikasi mesin motor dan mobil memang berbeda.
- `Mobil mobil = new Mobil();` → Membuat objek Mobil baru tanpa parameter menggunakan default constructor.
- `mobil.setBahanBakar("Pertamax");` → Mengisi atribut `bahanBakar` pada objek mobil dengan nilai "Pertamax" menggunakan setter.
- `mobil.setMesin(mesinMobil);` → Menyimpan objek `mesinMobil` ke dalam atribut `mesin` pada objek mobil menggunakan setter, menghubungkan objek Mesin dengan objek Mobil.
- `mobil.setJumlahPintu(4);` → Mengisi atribut `jumlahPintu` pada objek mobil dengan nilai 4. Atribut `jumlahPintu` merupakan 1 atribut tambahan yang

ditambahkan pada kelas Mobil sesuai ketentuan soal, selain atribut bahanBakar dan mesin yang sudah diwajibkan. Tipe data int sehingga tidak perlu tanda kutip.

- `System.out.println("DATA MOBIL");` → Mencetak teks header sebagai judul bagian data mobil.
- `System.out.println("Jumlah Pintu : " + mobil.getJumlahPintu());` → Mencetak jumlah pintu mobil. `mobil.getJumlahPintu()` memanggil getter untuk mengambil nilai atribut `jumlahPintu` yang bertipe int. Hasilnya tercetak Jumlah Pintu : 4.
- `System.out.println("Bahan Bakar : " + mobil.getBahanBakar());` → Mencetak jenis bahan bakar mobil menggunakan getter `getBahanBakar()`. Hasilnya tercetak Bahan Bakar : Pertamax.
- `System.out.println("Spesifikasi Mesin :");` → Mencetak teks sub-header sebagai penanda data spesifikasi mesin mobil.
- `System.out.println("Jumlah Silinder : " + mobil.getMesin().getJumlahSilinder());` → Menggunakan method chaining — `mobil.getMesin()` mengambil objek Mesin milik mobil, lalu `.getJumlahSilinder()` mengambil nilai silindernya. Hasilnya tercetak Jumlah Silinder : 4.
- `System.out.println("Kapasitas Silinder : " + mobil.getMesin().getKapasitasSilinder() + " CC");` → Menggunakan method chaining untuk mengambil kapasitas silinder mesin mobil, lalu ditambah teks " CC" sebagai satuan. Hasilnya tercetak Kapasitas Silinder : 1500.0 CC.
- `System.out.println("Langkah : " + mobil.getMesin().getLangkah());` → Mengambil jenis langkah mesin mobil melalui method chaining. Hasilnya tercetak Langkah : 4-Tak.

3. Output



```
C:\Windows\System32\cmd.exe
C:\Users\Bestdym\Data\Praktikum\OOP\TugasUTS>javac mesin/Mesin.java kendaraan/SepedaMotor.java kendaraan/Mobil.java
Utama.java
C:\Users\Bestdym\Data\Praktikum\OOP\TugasUTS>java Utama
Nama      : Dimas Nurdiansyah
NIM       : 251110058

DEALER JOGJA ISTIMEWA
=====
DATA SEPEDA MOTOR
Merk      : Yamaha
Bahan Bakar : Pertalite
Spesifikasi Mesin :
Jumlah Silinder : 1
Kapasitas Silinder : 150.0 CC
Langkah   : 2-Tak

DATA MOBIL
Jumlah Pintu : 4
Bahan Bakar : Pertamax
Spesifikasi Mesin :
Jumlah Silinder : 4
Kapasitas Silinder : 1500.0 CC
Langkah   : 4-Tak
```

Gambar 3. 1 Output Program

Penjelasan Cara Running Program :

Untuk menjalankan program, langkah pertama yang harus dilakukan adalah membuka Command Prompt (CMD) lalu navigasi ke folder tempat project disimpan menggunakan perintah:

```
cd C:\Users\Bestdym\Data\Praktikum\OOP\TugasUTS
```

Perintah cd (change directory) wajib dilakukan terlebih dahulu agar CMD mengetahui lokasi file program yang akan dikompilasi dan dijalankan. Sesuaikan path di atas dengan lokasi folder project yang ada di komputer masing-masing. Setelah berada di dalam folder TugasUTS, langkah selanjutnya adalah kompilasi, yaitu proses menerjemahkan kode Java menjadi file .class yang dapat dimengerti dan dijalankan oleh komputer. Terdapat dua cara kompilasi yang dapat dilakukan:

Cara pertama, kompilasi sekaligus :

```
javac mesin/*.java kendaraan/*.java Utama.java
```

Perintah ini mengompilasi seluruh file .java dalam satu baris sekaligus. Urutan penulisan file sangat penting Mesin.java harus dikompilasi paling pertama karena kelas SepedaMotor dan Mobil bergantung pada kelas Mesin. Jika urutan terbalik, Java akan mengeluarkan error karena belum mengenali kelas Mesin saat mengompilasi kelas yang membutuhkannya. Cara ini lebih cepat dan praktis karena cukup satu baris perintah untuk mengompilasi seluruh file sekaligus.

Cara kedua, kompilasi satu per satu:

```
javac mesin/Mesin.java
```

```
javac kendaraan/SepedaMotor.java
```

```
javac kendaraan/Mobil.java
```

```
javac Utama.java
```

Cara ini mengompilasi setiap file secara berurutan dari kelas yang paling atas hierarkinya hingga ke bawah. Urutan ini wajib diikuti karena setiap kelas bergantung pada kelas yang dikompilasi sebelumnya — misalnya SepedaMotor.java membutuhkan Mesin.java sudah ter-compile lebih dulu, dan Utama.java membutuhkan semua kelas lainnya sudah siap. Cara ini lebih direkomendasikan saat belajar karena jika terjadi error, pesan error hanya muncul dari file yang bermasalah sehingga lebih mudah dilacak dan diperbaiki. Setelah proses kompilasi selesai tanpa error, program dieksekusi menggunakan perintah:

```
java Utama
```

Perintah ini menjalankan class Utama yang di dalamnya terdapat method main() sebagai titik masuk eksekusi program. Program kemudian akan menampilkan output

berupa identitas pembuat program, header dealer, data lengkap sepeda motor beserta spesifikasi mesinnya, dan data lengkap mobil beserta spesifikasi mesinnya.

Penjelasan Output :

Output program terdiri dari dua bagian utama yaitu data sepeda motor dan data mobil.

Baris pertama menampilkan identitas pembuat program (Nama dan NIM).

Header "DEALER JOGJA ISTIMEWA" beserta garis = sebanyak 40 karakter dibuat menggunakan `"=".repeat(40)` — method bawaan Java untuk mengulang karakter sebanyak N kali.

DATA SEPEDA MOTOR menampilkan:

- Merk : Yamaha → diambil dari `motor.getMerk()`
- Bahan Bakar : Pertalite → diambil dari `motor.getBahanBakar()`
- Jumlah Silinder : 1 → diambil dari `motor.getMesin().getJumlahSilinder()`, yaitu mengakses objek mesin di dalam motor, lalu mengambil jumlah silindernya
- Kapasitas Silinder : 150.0 CC → diambil dari `motor.getMesin().getKapasitasSilinder()`, nilai bertipe double sehingga tampil sebagai 150.0
- Langkah : 2-Tak → diambil dari `motor.getMesin().getLangkah()`

DATA MOBIL menampilkan:

- Jumlah Pintu : 4 → diambil dari `mobil.getJumlahPintu()`, bertipe int
- Bahan Bakar : Pertamax → diambil dari `mobil.getBahanBakar()`
- Jumlah Silinder : 4 → diambil dari `mobil.getMesin().getJumlahSilinder()`
- Kapasitas Silinder : 1500.0 CC → diambil dari `mobil.getMesin().getKapasitasSilinder()`
- Langkah : 4-Tak → diambil dari `mobil.getMesin().getLangkah()`

4. Kesimpulan

Program yang telah dibuat merupakan implementasi konsep dasar Pemrograman Berorientasi Objek (PBO) menggunakan bahasa Java. Program ini terdiri dari empat kelas yang diorganisir ke dalam dua package yaitu package mesin yang berisi kelas Mesin, dan package kendaraan yang berisi kelas SepedaMotor dan Mobil, serta satu kelas Utama yang berada di default package sebagai titik masuk program.

Konsep utama yang diterapkan dalam program ini adalah Enkapsulasi, yaitu dengan menjadikan seluruh atribut pada setiap kelas bersifat private sehingga data tidak dapat diakses secara langsung dari luar kelas. Akses terhadap data tersebut hanya dapat dilakukan melalui method getter untuk membaca nilai dan setter untuk mengubah nilai, sehingga data lebih terlindungi dan terstruktur dengan baik.

Selain enkapsulasi, program ini juga menerapkan konsep Object Composition yaitu kelas SepedaMotor dan Mobil menggunakan objek dari kelas Mesin sebagai salah satu atributnya. Hal ini memungkinkan setiap kendaraan memiliki spesifikasi mesin yang berbeda-beda tanpa perlu mendefinisikan ulang atribut mesin di setiap kelas. Dengan pemisahan kelas ke dalam package yang sesuai, kode program menjadi lebih terorganisir, mudah dipahami, dan mudah dikembangkan di masa mendatang.